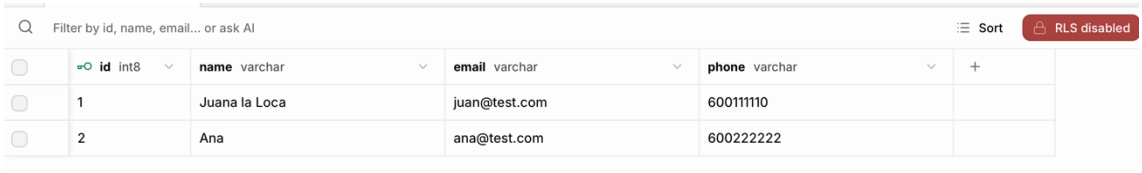
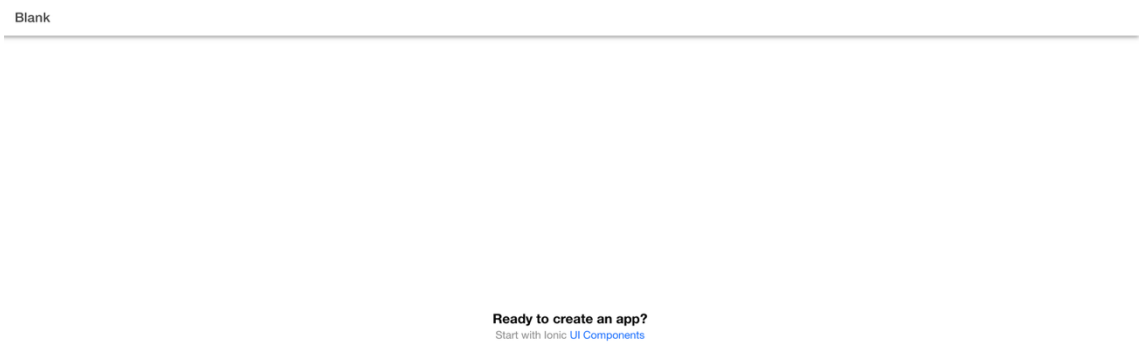
Captura 1 - Tabla clients de SupaBase de Table editor:



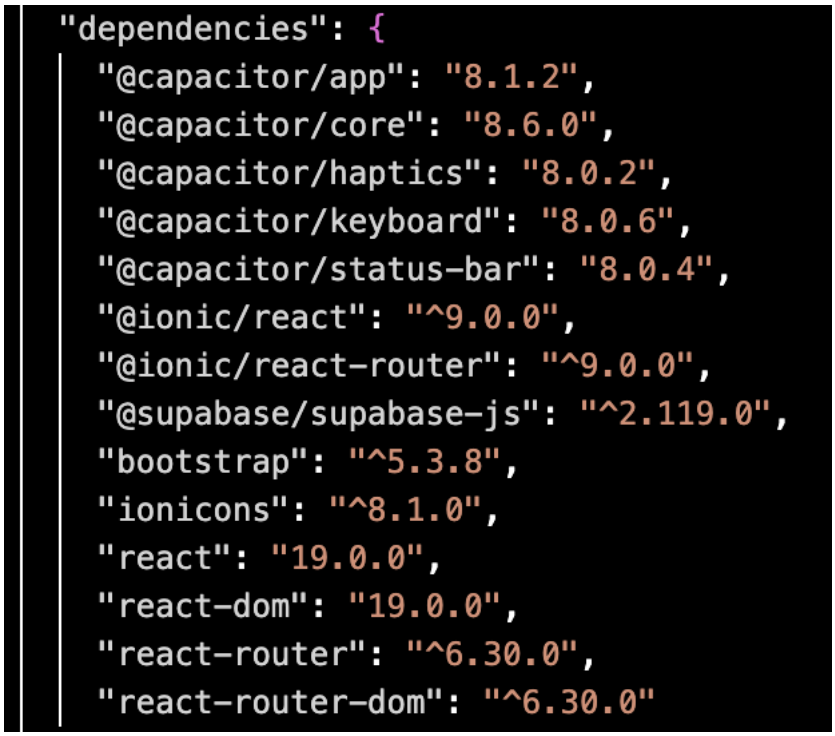
Filter by id, name, email... or ask AI — Sort — RLS disabled

id int8	name varchar	email varchar	phone varchar
1	Juana la Loca	juan@test.com	600111110
2	Ana	ana@test.com	600222222

Captura 2 - Proyecto app-clients renderizado en Port 8100:



Captura 3 - Versiones de librerías supabase y boostrap:



```
"dependencies": {
  "@capacitor/app": "8.1.2",
  "@capacitor/core": "8.6.0",
  "@capacitor/haptics": "8.0.2",
  "@capacitor/keyboard": "8.0.6",
  "@capacitor/status-bar": "8.0.4",
  "@ionic/react": "^9.0.0",
  "@ionic/react-router": "^9.0.0",
  "@supabase/supabase-js": "^2.119.0",
  "bootstrap": "^5.3.8",
  "ionicons": "^8.1.0",
  "react": "19.0.0",
  "react-dom": "19.0.0",
  "react-router": "^6.30.0",
  "react-router-dom": "^6.30.0"
```

Captura 4 - Fichero supabase.ts de configuración + explicar técnicamente por qué debes usar VITe en lugar de REACT. *Vite es una plataforma especial?

Vite no sustituye a React: es la herramiento que permite ejecutar y compilar React. React define la interfaz, Vite genera y sirve la aplicación.

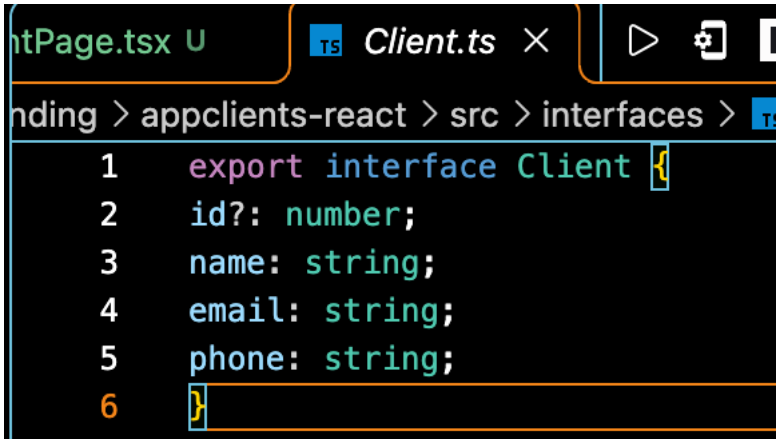
```ts
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;

if (!supabaseUrl || !supabaseKey) {
  throw new Error(
    'Configura VITE_SUPABASE_URL y VITE_SUPABASE_PUBLISHABLE_KEY en el archivo .env de la raíz del proyecto.',
  );
}

export const supabase = createClient(supabaseUrl, supabaseKey);
```

Captura 5 - Interfaz cliente. Verifica que los atributos coinciden con los campos de tabla. También tipo de dato. Explíca que aporta ? Al declarar un atributo:



Captura 6 - Método para añadir clientes en services:

```ts
static async addClient(client: Client): Promise<void> {
  const { error } = await supabase.from('clients').insert([
  if (error) {
    throw error;
  }
}
```

Captura 7 - Del App.tsx del RouterOutlet. Hecho todos hasta aquí:

App.tsx

ionicBinding > appclients-react > src > App.tsx > App

```tsx
import '@ionic/react/css/palettes/dark.system.css';

/* Theme variables */
import './theme/variables.css';

// Boostrap
import 'bootstrap/dist/css/bootstrap.min.css'

setupIonicReact();

const App: React.FC = () => (
  <IonApp>
    <IonReactRouter>
      <IonRouterOutlet>
        <Route path="/home" element={<Home />} />
        <Route path="/clients" element={<ClientsPage />} />
        <Route path="/" element={<Navigate to="/home" replace />} />
      </IonRouterOutlet>
    </IonReactRouter>
  </IonApp>
);

export default App;
```

Captura 8 - Listado de clientes renderizado:



Captura 9 - Dentro de clientspage, captura la “signal” de React con el hook useState. En react no hay signals:

```tsx
export default function ClientsPage(): JSX.Element {
  const [clients, setClients] = useState<Client[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

Captura 10 - Captura del error de ionic generate pages/ NewClientPage. Qué comando permite que, en ionic react, generar páginas? Se pueden crear de alguna manera entonces?

En Ionic/Angular no existe el comando. Tendrías que crearlo manualmente.

```
mateofernandezblasco@MacBook-Air-de-Mateo appclients-react % ionic generate pages/  NewClientPage
[ERROR] Cannot perform generate for React projects.

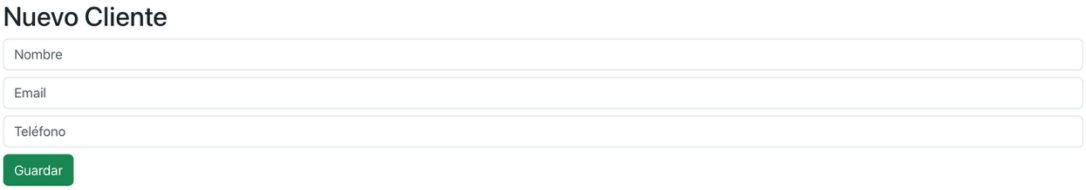
        Since you're using the React project type, this command won't work. The Ionic CLI doesn't know how to generate
        framework components for React projects.
```

Captura 11 - Qué significan los "…" en NewClientsPage.tsx:

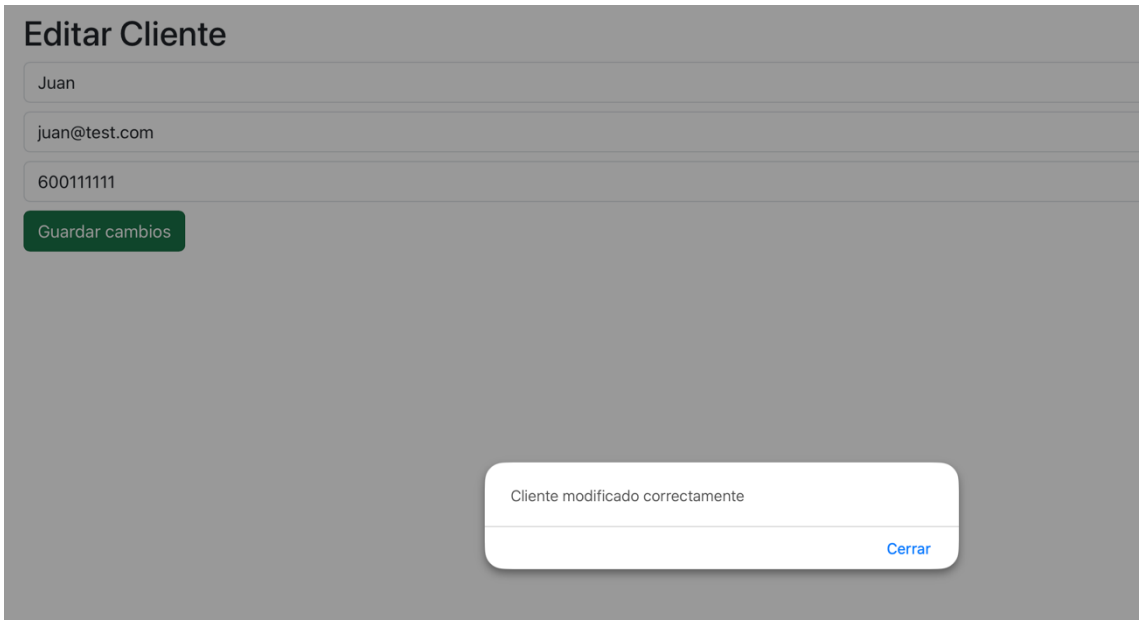
Los "…" sirven para "copiar" un objeto y modificar solo una parte sin perder el resto.

```
return (
  <div className="container mt-4">
    <h2>Nuevo Cliente</h2>
    <input
      className="form-control mb-2"
      placeholder="Nombre"
      value={client.name}
      onChange={(e) => setClient({ ...client, name: e.target.value })}
    />
    <input
      className="form-control mb-2"
      placeholder="Email"
      type="email"
      value={client.email}
      onChange={(e) => setClient({ ...client, email: e.target.value })}
    />
    <input
      className="form-control mb-2"
      placeholder="Teléfono"
      type="tel"
      value={client.phone}
      onChange={(e) => setClient({ ...client, phone: e.target.value })}
    />
    <button className="btn btn-success" onClick={save}>
      Guardar
    </button>
```

Captura 12 - Muestra el formulario para añadir clientes:



Captura 13 - Evidencia de actualizar cliente:



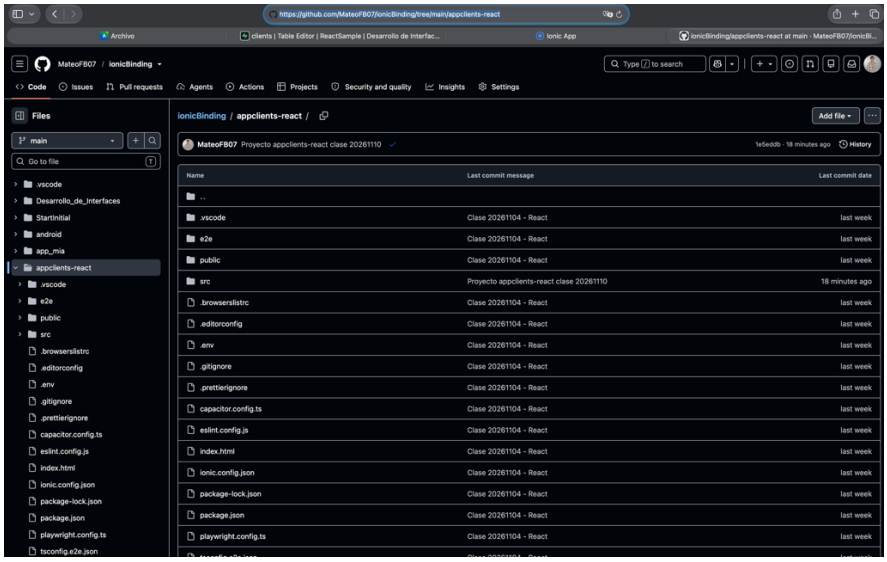
Captura 14 - Evidencia de eliminar cliente:



Captura 15 - Captura de ruta paramétrica:



Captura 16: Enlace de GitHub de tu proyecto:



Captura 17: Enlace de Vercel:

